Security Review Report
NM-0983 - ETHGas



NETHERMIND
SECURITY

(October 15, 2026)

Contents

1 Executive Summary **2**

2 Audited Files **3**

3 Summary of Issues **3**

4 System Overview **4**
4.1 Execution flow . . . 4
4.2 Validator Registration Modes . . . 4

5 Risk Rating Methodology **5**

6 Issues **6**
6.1 [Medium] Error logs expose replayable login credentials and refresh tokens . . . 6
6.2 [Low] SSV environment variable names do not match the documentation . . . 6
6.3 [Low] Response parsing errors can expose authentication tokens in logs . . . 6
6.4 [Low] Result code 1 from the batch verification is not handled . . . 7
6.5 [Low] Unchecked signing can bind validators to an unintended account . . . 7
6.6 [Low] Incorrect log and documentation for the builder removal . . . 7
6.7 [Low] Rejected configuration updates do not halt validator operation . . . 8
6.8 [Info] Ownership verification errors are handled differently across DVT modes . . . 8

7 Documentation Evaluation **9**

8 Test Suite Evaluation **10**
8.1 Test Coverage . . . 10
8.2 Test Coverage . . . 10

9 About Nethermind **11**

1 Executive Summary

This document presents the results of the security review conducted by Nethermind Security for the ETHGas Preconf Commit-Boost Module. ETHGas operates an Ethereum blockspace marketplace where validators sell block capacity in advance as blockspace futures and inclusion preconfirmations, converting uncertain MEV into contracted revenue, while traders and searchers buy guaranteed inclusion.

The reviewed repository is the supply-side connector that a node operator runs to participate in that market, linking validator infrastructure to the ETHGas Exchange and its relays through a set of Commit-Boost modules.

The audit comprises 2931 lines of Rust code and a TOML file consisting of 78 lines. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 8 points of attention, where 1 is classified as `Medium`, 6 are classified as `Low`, and 1 is classified as `Informational` or `Best Practices` severity. The issues are summarized in Fig. 1. The issue is summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

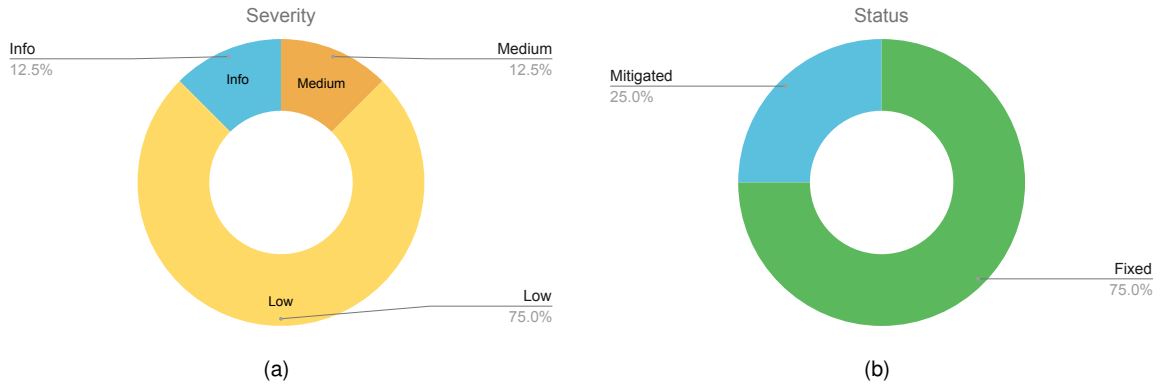


Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (1), **Low** (6), **Undetermined** (0), **Informational** (1), **Best Practices** (0). **Distribution of status: Fixed** (6), **Acknowledged** (0), **Mitigated** (2), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	September 18, 2026
Final Report	October 15, 2026
Initial Commit	`8b82f37`
Final Commit	`b3832c2`
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	bin/ethgas_commit.rs	1382	75	5.4%	99	1556
2	bin/ethgas_deposit.rs	479	42	8.8%	42	563
3	bin/gen_jwt.rs	36	1	2.8%	4	41
4	src/lib.rs	6	0	0.0%	0	6
5	src/obol_registry.rs	461	16	3.5%	27	504
6	src/login_types.rs	59	6	10.2%	7	72
7	src/ofac.rs	69	2	2.9%	8	79
8	src/query_pubkey.rs	238	19	8.0%	29	286
9	src/dvt_types.rs	23	6	26.1%	4	33
10	src/utils.rs	178	8	4.5%	26	212
	Total	**2931**	**175**	**6.0%**	**246**	**3352**

3 Summary of Issues

	Finding	Severity	Update
1	Error logs expose replayable login credentials and refresh tokens	Medium	Fixed
2	SSV environment variable names do not match the documentation	Low	Fixed
3	Response parsing errors can expose authentication tokens in logs	Low	Fixed
4	Result code 1 from the batch verification is not handled	Low	Mitigated
5	Unchecked signing can bind validators to an unintended account	Low	Fixed
6	Incorrect log and documentation for the builder removal	Low	Fixed
7	Rejected configuration updates do not halt validator operation	Low	Mitigated
8	Ownership verification errors are handled differently across DVT modes	Info	Fixed

4 System Overview

The ETHGas Commit-Boost module acts as an orchestration layer between a Commit-Boost validator deployment and the ETHGas Exchange. Its main responsibilities are to establish an authenticated Exchange session, apply account-level configuration, associate validator consensus public keys with an ETHGas account, manage validator registrations across standard and DVT-based deployment modes, query registered validator state, and fund the collateral account.

The implementation is organized around three executable entry points. `ethgas_commit` is the main orchestration binary and coordinates the authentication, account-configuration, validator-registration, and query flows. `ethgas_deposit` implements the separate collateral deposit and account-transfer workflow. `gen_jwt` is a bootstrap utility that generates the shared authentication material required by the Commit-Boost module and `cb_signer`; it is used during setup rather than as part of the regular registration workflow.

The supporting library modules separate the remaining responsibilities: EIP-712 message handling and signing, DVT-specific message types, SSV and Obol operator handling, OFAC and payout updates, and registered-validator queries.

4.1 Execution flow

During execution, `ethgas_commit` loads the Commit-Boost configuration and authenticates to the ETHGas Exchange using an EOA-signed EIP-712 challenge or supplied JWTs. It then applies account settings, including pricer and builder delegations, collateral per slot, and validator mode, before processing the selected registration mode. Execution can run once or repeat at a configured interval.

Standard registration obtains validator public keys from `cb_signer`, while `standard-mux` selects them from the Commit-Boost mux configuration. Both use BLS signatures produced by `cb_signer` to authorize association with the ETHGas account. SSV and Obol registration instead require separate operator-owner verification before managing their associated validators; the Obol flow also refreshes the Exchange's cluster state. The module supports registration and deregistration, applies OFAC and payout settings following registration, and can export registered validator state.

Collateral funding is handled separately by `ethgas_deposit`. It authenticates to the Exchange, checks that the Exchange-provided pool address matches the configured contract, and deposits native ETH using the Ethereum RPC endpoint from the PBS configuration. After transaction confirmation, it queries the Exchange deposit history and transfers the configured amount from the current account to the trading account.

4.2 Validator Registration Modes

The module supports three primary registration modes, selected through `registration_mode`:

Standard: Used for non-DVT validators. The module obtains validator public keys from the configured signer and requests BLS signatures to associate them with the ETHGas account. The `standard-mux` variant restricts registration to validators selected by ETHGas-related entries in the Commit-Boost mux configuration.

SSV: Used for validators operated through SSV. Registration requires proof of ownership of an associated SSV node operator, using a private key, keystore, Ledger, or an on-chain verification transaction. Validator public keys can be explicitly configured or discovered through the SSV API.

Obol: Used for validators operated through Obol. Registration similarly requires operator-owner verification, with the corresponding configuration using the `obol_` prefix. Both DVT modes allow registration without the validator signer setup required by standard mode.

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Medium] Error logs expose replayable login credentials and refresh tokens

File(s): bin/ethgas_commit.rs, src/obol_registry.rs, bin/ethgas_deposit.rs

Description: Some of the calls, like verify/batch2 or update/ofac send their parameters in a form body, but the credential-bearing calls use the query string instead. The reqwest::Error stores the request URL and prints it in both Display and Debug, so a failed request writes the credential to the log at ERROR level. Being ERROR level, it cannot be filtered out, and logs tend to be readable by more parties than the host itself: forwarding agents, central log stores and whoever has access to them, backups, or another account on the machine. It is also, being a login failure, the line an operator is most likely to paste into a support channel while asking why. Exposed this way:

- /login/verify — logs addr, nonceHash and signature at error!(?err, "Service failed"). A failed request leaves the nonce unspent, so replaying the three as they are returns a token for the operator's account, and /validator/deregister and /validator/update/validatorPayoutAddress then need only that token: the first withdraws every validator the operator has registered and ends their preconf income, the second redirects their payouts to an address of the caller's choosing.
- /login/refresh — the refresh token. It mints access tokens on demand, so it yields the same account takeover as above.
- /ssv/operator/verify and /obol/operator/verify — the owner's ownership signature. Not replayable, because the signed message names the operator's own ETHGas user. It would matter only if the exchange took the account from the caller's session instead, in which case the holder could claim the operator's node operator owner and register its validators to their own account.

Recommendation(s): Send these values in the request body rather than the query string, as verify/batch2 already does. This requires the exchange to accept them in a body at /login/verify, /login/refresh, /ssv/operator/verify and /obol/operator/verify.

Status: Fixed

Update from the client: Fixed in commit 4baecc1.

6.2 [Low] SSV environment variable names do not match the documentation

File(s): bin/ethgas_commit.rs, src/obol_registry.rs

Description: The code reads two names, the second misspelled:

```
let keystore_paths = env::var("SSV_NODE_OPERATOR_OWNER_KEYSTORES");
let password_paths = env::var("SSV_NODE_OPERATOR_OWNER_PASSOWRDS");
```

extra-config-notes.md tells operators to set SSV_NODE_OPERATOR_OWNER_KEYSTORE_PATHS and SSV_NODE_OPERATOR_OWNER_PASSWORD_PATHS, so neither name matches. An operator following the documentation falls to the error arm and is told ssv_node_operator_owner_keystores cannot be empty, naming a config field they did not use.

Recommendation(s): Correct the spelling and align both names with the documentation.

Status: Fixed

Update from the client: Fixed in commit 22a7a66.

6.3 [Low] Response parsing errors can expose authentication tokens in logs

File(s): bin/ethgas_commit.rs

Description: The body returned by /login/verify carries the access token, and is parsed with serde_json::from_str(...).expect(...). serde_json quotes the offending value in its type errors, in both Display and Debug, and expect prints that error, so when the mismatch falls where the token sits the token is written out verbatim. It goes to stdout and to the log file, and stays there for the retention period.

Not every malformed response discloses it. A renamed or absent field yields missing field "accessToken" with no value attached. The token only appears when the exchange returns it at a position the module expects to be something else, such as accessToken as a bare string rather than an object. The refresh response at line 1028 has the same shape and reaches error!(?err, ...), where the refresh token can print the same way.

Recommendation(s): Do not format parse errors for bodies that carry credentials, log the status code and body length instead of the serde_json error.

Status: Fixed

Update from the client: Fixed in commit 4baecc1.

6.4 [Low] Result code 1 from the batch verification is not handled

File(s): bin/ethgas_commit.rs

Description: The `verify/batch2` response maps each public key to a result code. The module filters for three of them and reports each:

```
if verify_result.result == 0 { ... }   // registered
if verify_result.result == 3 { ... }   // previously registered
if verify_result.result == 2 { ... }   // invalid signature
```

The published enum also defines `1`, which no branch covers. A key returned with that code lands in none of the three lists, so it appears in no log line and in no counter, and the operator would have to compare the number of keys submitted against `newly_registered_key_num` to notice anything was refused.

Recommendation(s): Consider handling `1` explicitly.

Status: Mitigated

Update from the client: For code 1 from `verify/batch2`, the backend no longer return code 1 so there is no need to handle this case.

6.5 [Low] Unchecked signing can bind validators to an unintended account

File(s): bin/ethgas_commit.rs, src/utils.rs

Description: The message the validator BLS keys sign is `RegisteredInfo { eoa_address }`, the ETHGas account those validators are bound to. The address arrives in the `validator/register` response and is signed exactly as received:

```
let info = RegisteredInfo {
    eoa_address: api_validator_request_response_data_message.eoa_address, // from the response
};
let request = SignConsensusRequest::builder(pubkey.clone()).with_msg(&info);
let signature = self.config.signer_client.request_consensus_signature(request).await?;
```

The module never looks at the value. The operator has no opportunity to confirm the address before their keys sign. The SSV and Obol paths do the same with `userAddress` inside the server-supplied `messageToSign`, signed unchecked at src/utils.rs.

Recommendation(s): Consider adding the expected ETHGas account address to the module config. Unpack the address from the response, and from `messageToSign` in the DVT paths, compare it against the configured value, and sign only on a match. On a mismatch, abort and report both addresses so the operator can see what the exchange asked them to sign.

Status: Fixed

Update from the client: Fixed in commit 81b8f68.

6.6 [Low] Incorrect log and documentation for the builder removal

File(s): bin/ethgas_commit.rs, extra-config-notes.md

Description: Whether to contact the delegation endpoint is decided by the presence of `builder_pubkey`, not by `enable_builder`:

```
match &self.config.extra.builder_pubkey {
    Some(builder_pubkey) => {
        exchange_api_url = Url::parse(&format!(
            "{}{}{}{}",
            self.config.extra.exchange_api_base,
            "/api/v1/user/delegate/builder?enable=",
            self.config.extra.enable_builder,
            "&publicKeys=",
            builder_pubkey
        ))?;
        res = client.post(exchange_api_url.to_string())
            ...
            .send()
            .await?;
        ...
    }
    None => {
        info!("builder delegation is disabled and builder_pubkey is not set");
    }
}
```

The log emitted when the builder public key is absent states an incorrect fact, since no builder was disabled. The documentation (`extra-config-notes.md`) tells operators that `builder_pubkey` "can be omitted when `enable_builder = false`", but this is also incorrect, since an absent builder public key results in a no-op. To actually disable the builder, the builder public key must be provided and the `enable_builder` flag set to `false`.

Recommendation(s): Consider fixing the log and the documentation so that the operator is given correct information about disabling the builder.

Status: Fixed

Update from the client: Fixed in commit b2c02ae, documentation has been updated.

6.7 [Low] Rejected configuration updates do not halt validator operation

File(s): src/ofac.rs, src/utils.rs, bin/ethgas_commit.rs, bin/ethgas_commit.rs, bin/ethgas_commit.rs, src/utils.rs

Description: Six settings are applied per pass: OFAC, payout address, collateral, pricer and builder delegation, and validator mode. All six treat a rejection from the exchange the same way, logging it and then carrying on as if it had applied:

```
// src/ofac.rs
} else if enable_ofac {
    error!("failed to enable ofac: {}", res_json.error_msg_key.unwrap_or_default());
}
// ...
Ok(())   // Returned whether or not the setting applied
```

Some configuration failures carry greater weight than others and may require more than emitting an error log. If enabling OFAC fails, the validator keeps accepting blocks containing sanctioned transactions while the operator believes it has opted out, which is a compliance exposure rather than a missing preference. Failures of this kind need treatment beyond an error log, since the lack of a correct configuration can lead to serious consequences.

Recommendation(s): Consider adding further measures for failures of important configuration fields. These can include retrying the call and an additional mechanism to inform and protect the user.

Status: Mitigated

Update from the client: Addressed in commit b2c02ae, OFAC config failures will cause error, other rejected configs remain unchanged.

6.8 [Info] Ownership verification errors are handled differently across DVT modes

File(s): bin/ethgas_commit.rs `key`/`keystore`/`ledger` branch, bin/ethgas_commit.rs `tx` branch

Description: Each DVT mode proves control of the node operator owner address before registering that owner's validators. The first ownership call is `operator/register` in `key`, `keystore` and `ledger` modes, and `operator/verifyByTx` in `tx` mode. The same rejection is treated differently:

```
// key / keystore / ledger, :788
false => { return Err(std::io::Error::other("failed to get the SSV node operator
                    registration message for signing").into()); }

// tx, :747
} else { error!("failed to register ssv node operator owner address: {}", ...); }
```

So an operator who cannot prove ownership sees the pass abort in three modes and continue in the fourth. Changing `ssv_node_operator_owner_mode` silently changes what a failure means, and in `tx` mode execution reaches `/operator/validator/register` at bin/ethgas_commit.rs for an owner the exchange has just declined.

Recommendation(s): Verify whether the different behavior on failures across modes is intended and in accordance with the exchange logic.

Status: Fixed

Update from the client: Fixed in commit b2c02ae.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about EthGas's documentation

The ETHGas team provided an overview of the core system components during our meetings, presenting a clear explanation of the intended functionalities and addressing questions raised by the Nethermind Security team. These sessions offered valuable insights and ensured a comprehensive understanding of the project's technical foundations.

The repository features a `README.md` document which provides an overview of the three components, node operator guides, and instructions on how to build the project. A very detailed markdown document named `extra-config-notes.md` is also provided, covering the startup processes, troubleshooting, and a variety of configuration options with expected values and explanations.

8 Test Suite Evaluation

8.1 Test Coverage

	File	Functions	Lines	Regions
1	bin/ethgas_commit.rs	58/60	1219/1267	1443/1548
2	bin/ethgas_deposit.rs	24/24	379/388	420/452
3	bin/gen_jwt.rs	4/4	31/31	76/76
4	src/login_types.rs	3/3	15/17	15/23
5	src/obol_registry.rs	15/15	344/351	463/481
6	src/ofac.rs	2/2	52/52	81/81
7	src/query_pubkey.rs	12/12	180/180	262/267
8	src/utils.rs	22/22	285/285	364/366
Total		**140/142**	**2505/2571**	**3124/3294**

8.2 Test Coverage

```
test tests::truncated_sensitive_body_has_safe_diagnostics ... ok
test tests::mode_api_rejection_with_valid_schema_is_distinct_from_bad_json ... ok
test tests::sensitive_response_errors_do_not_disclose_secrets ... ok
test tests::export_requests_reject_bad_urls_and_operator_transport_failures ... ok
test tests::exports_write_exact_keys_for_each_operator_and_propagate_io_failures ... ok
test tests::exports_handle_empty_failed_and_malformed_responses ... ok
test tests::signatures_recover_against_independent_eip712_encoding ... ok
test tests::api_helpers_preserve_their_error_policies ... ok
test tests::signing_rejects_every_untrusted_domain_field_and_dvt_user ... ok
test tests::batched_settings_route_and_encode_every_mode_and_boundary ... ok
test tests::arbitrary_messages_cannot_bypass_the_required_schema ... ok
test tests::legacy_logging_preserves_safe_library_diagnostics ... ok
test result: ok. 12 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.20s

test tests::collateral_validation_covers_bounds_precision_and_malformed_values ... ok
test tests::optional_settings_deserialize_without_resetting_values ... ok
test tests::signing_id_selects_exact_module_and_rejects_invalid_files ... ok
test tests::signing_id_environment_errors_do_not_mutate_the_parent_environment ... ok
test tests::refresh_boundary_and_new_token_usage ... ok
test tests::omitted_updates_send_no_requests_and_query_defaults_to_enabled ... ok
test tests::authentication_transport_errors_are_redacted_and_stop_follow_up_calls ... ok
test tests::failed_registration_settings_propagate_and_stop_later_updates ... ok
test tests::response_rejections_and_missing_password_files_are_handled ... ok
test tests::malformed_login_and_operator_challenges_fail_before_verification ... ok
test tests::login_rejects_failed_verification_and_invalid_cookies ... ok
test tests::standard_registration_handles_rejection_mismatch_and_mixed_results ... ok
test tests::builder_configuration_matrix_and_explicit_updates ... ok
test tests::operator_configuration_errors_and_failed_verification_stop_registration ... ok
test tests::operator_response_variants_and_transaction_verification ... ok
test tests::operator_registration_and_deregistration_cover_empty_and_chunked_keys ... ok
test tests::transport_errors_stop_each_registration_and_settings_phase ... ok
test tests::keystores_with_invalid_scalar_cannot_register_operators ... ok
test tests::encrypted_keystores_trim_password_files_and_fail_closed ... ok
test tests::standard_registration_batches_and_signer_requests_match ... ok
test tests::legacy_logging_preserves_safe_registration_diagnostics ... ok
test result: ok. 21 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 4.66s

test tests::confirmation_timeout_and_revert_are_errors ... ok
test tests::contract_mismatch_and_malformed_contract_response_prevent_submission ... ok
test tests::invalid_signers_transport_and_pending_errors_are_reported ... ok
test tests::reverted_rpc_receipt_prevents_off_chain_transfer ... ok
test tests::oversized_amounts_and_history_quantities_return_errors_without_panics ... ok
test tests::deposit_submits_exact_value_and_transfers_between_expected_accounts ... ok
test tests::login_validates_domains_and_verification_results ... ok
test tests::exchange_transport_failures_stop_deposit_and_transfer ... ok
test tests::malformed_history_and_missing_accounts_prevent_transfer ... ok
test tests::local_evm_receives_exact_deposit_and_rejects_reverting_contract ... ok
test tests::history_and_transfer_response_variants_are_handled ... ok
test tests::legacy_logging_preserves_safe_deposit_diagnostics ... ok
test result: ok. 12 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.07s

running 10 tests
test generated_secrets_have_expected_mapping_and_io_errors_are_visible ... ok
test mux_registration_selects_only_ethgas_mux_and_relay_keys ... ok
test missing_password_and_deposit_rpc_configuration_are_visible_errors ... ok
test commit_cli_loads_config_login_or_supplied_tokens_and_runs_once ... ok
test deposit_cli_completes_with_config_or_environment_key_and_rejects_bad_inputs ... ok
test commit_cli_rejects_invalid_credentials_config_and_collateral ... ok
test repeated_runs_reload_config_without_reinitializing_metrics ... ok
test environment_keystores_reject_invalid_scalars_and_corrupt_files ... ok
test operator_environment_keys_and_keystores_validate_and_register ... ok
test cli_retries_login_failures_and_rejects_invalid_pbs_configuration ... ok
test result: ok. 10 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 5.14s
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.